

AI Models Suite Training, Datasets Repository & MATLAB Deployment Manual

A Definitive Blueprint covering the 3 Core AI Models (2D YOLOv8 Vision, 3D LiDAR PointPillars, and Seq2Seq GRU Trajectory Predictor), Full Datasets Links, Sizes, GPU Training Commands, and MATLAB ONNX Integration.

1. The 3 Core AI Models Architecture & Detailed Roles

Model Name & Architecture	Sensor Input	Specific Detection / Prediction Objective	Deployment Format
Model 1: 2D Vision Perception YOLOv8m / YOLO11m (PyTorch Ultralytics)	Front Windshield RGB Camera (60m Range)	Detects and classifies 7 Indian Traffic Classes : • Auto-rickshaws • Stray Cattle/Cows • Pedestrians • Two-wheelers / Bikes • Potholes & Cracks • Trucks/Buses • Cars	PyTorch (.pt) ONNX (best.onnx)
Model 2: 3D LiDAR Perception PointPillars / CenterPoint (Point Cloud NN)	Roof 360° LiDAR (80m, 32-beam)	Processes 3D point cloud clusters (X,Y,Z,Intensity) to estimate 3D Bounding Boxes (X, Y, Z, length, width, height) and precise obstacle distances.	3D BBoxes Point Clusters
Model 3: Motion Prediction Seq2Seq GRU / Social-LSTM (Recurrent Neural Net)	Past 1.5–2.0s Tracker Positions	Forecasts future non-lane trajectory paths over 1.0 to 3.0s horizon for erratic cattle crossing, pedestrians, and weaving bikes.	PyTorch (.pth) ONNX (trajectory.onnx)

2. Official Datasets Repository, Direct Links & Download Sizes

Dataset Name	Target AI Model	Direct Download / Source Link	Size	Dataset Content & Format
Roboflow Indian Traffic & Cattle Suite ■ (Recommended / Fast)	Model 1 (YOLOv8 Vision)	universe.roboflow.com/search?q=indian+traffic universe.roboflow.com/search?q=indian+cattle	~350 MB	5,000+ pre-annotated Indian road images in YOLO format (.txt bboxes) for Auto, Cow, Bike, Pothole.
Indian Driving Dataset (IDD) (IIIT Hyderabad Benchmark)	Model 1 (YOLOv8 Vision)	idd.insaan.iiit.ac.in	~12 GB	40,000+ high-density Indian traffic frames from Bangalore/Hyderabad with unstructured roads and dense obstacles.
Roboflow Pothole & Road Anomaly Dataset	Model 1 (YOLOv8 Vision)	universe.roboflow.com/search?q=pothole	~180 MB	2,500+ annotated images of asphalt road cracks, deep potholes, and surface irregularities.
nuScenes Mini Suite (LiDAR + Camera) ■ (Recommended LiDAR)	Model 2 (3D LiDAR)	nuscenes.org/download (v1.0-mini.tgz)	~3.8 GB	Calibrated 360° LiDAR 32-beam point clouds (.pcd) with 3D ground truth bounding boxes.
KITTI 3D Vision Benchmark	Model 2 (3D LiDAR)	cvlibs.net/datasets/kitti/eval_object.php?obj_benchmark=3d	~12 GB	Velodyne 64-beam 3D spatial point clouds and stereo camera calibrated benchmark.
Built-in Indian Trajectory Generator ■ (Fastest / Zero Download)	Model 3 (Trajectory Predictor)	Included in Repo Script: training_suite/train_trajectory_predictor.py	0 MB (Instant)	Generates 5,000+ non-lane trajectory paths (weaving two-wheelers, cattle crossing, pedestrian darting).
INTERACTION Dataset	Model 3 (Trajectory Predictor)	interaction-dataset.com	~2.5 GB	Unstructured roundabout, merging and multi-agent interaction motion tracks.

3. Step-by-Step AI Models Training Execution

A. Training Model 1: 2D Vision Object Detector (YOLOv8 / YOLO11)

```
# 1. Navigate to training suite and execute YOLO training
cd d:\SIH2026\training_suite
python train_yolo_detector.py
```

Configured Hyperparameters:

- Base Weights: yolov8m.pt (Transfer Learning with pre-trained COCO weights)
- Epochs: 50 | Batch Size: 16 (or 8 for 4GB RTX 2050) | Image Resolution: 640x640
- Augmentations: Mosaic=1.0, Flipud=0.5, HSV Color Jitter for sunny/rainy Indian roads
- Export Output: Generates best.pt and MATLAB-compatible best.onnx

B. Training Model 2 & 3: 3D LiDAR & Trajectory Predictor Neural Nets

```
# 2. Train Multi-Agent Seq2Seq GRU Trajectory Predictor
python train_trajectory_predictor.py
Saves: trajectory_model.pth and trajectory_predictor.onnx (Loss: ADE < 0.35m)
```

```
# 3. Train 3D LiDAR Point Cloud Cluster Detector
cd d:\SIH2026\demo_training_suite
python 02_train_lidar_detector.py      # Extracts 3D Bounding Box spatial depths
```

4. Post-Training Validation Benchmarks & Verification

Evaluation Metric	Target Benchmark	Quality Assessment & SIH Evaluation Criterion
mAP@0.50 (Mean Average Precision)	> 0.85 (85%+)	■ Production Ready: Reliable detection of Auto-rickshaws, cattle, pedestrians and potholes under harsh Indian lighting.
mAP@0.50:0.95 (Strict IoU)	> 0.55 (55%+)	Tight spatial bounding-box localization around obstacle borders.
Precision & Recall	> 0.80 (80%+)	Zero false alarms (avoids phantom emergency braking) and zero missed obstacles.
Trajectory Prediction ADE	< 0.35 meters	Average Displacement Error well within 1.5m clearance margin, enabling safe Frenet replanning.

5. Loading & Executing ONNX Models in MATLAB / Simulink

```
% =====
% MATLAB COMMAND WINDOW / SCRIPT: LOADING TRAINED AI MODELS
% =====
% 1. Import Trained 2D YOLOv8 Detector into MATLAB Deep Learning Toolbox
net_yolo = importONNXNetwork('d:/SIH2026/training_suite/best.onnx');

% 2. Import Multi-Agent Trajectory Prediction Neural Network
net_trajectory = importONNXNetwork('d:/SIH2026/training_suite/trajectory_predictor.onnx');

% 3. Real-time Closed-Loop Perception & Prediction Loop
cameraFrame = getCameraImage(egoVehicle);           % 3D Synthetic / Real Frame
detections = predict(net_yolo, cameraFrame);         % YOLO Inference (Auto/Cow/Pothole)
futurePaths = predict(net_trajectory, pastHistory);  % 1-3s Motion Forecast for Frenet Planner
```

6. Google Colab Cloud GPU Training & Best Practices

- **Zero-Install Cloud Training (Google Colab / Kaggle):** If you prefer not to load your laptop GPU, use Google Colab with free NVIDIA T4 (16GB VRAM) for 5x faster training. Download generated best.onnx directly.
- **Local Laptop GPU Optimization (RTX 2050 4GB):** Set batch=16 for YOLOv8n/s, or batch=8 for YOLOv8m to prevent CUDA Out-of-Memory (OOM).
- **Real-time Inference Speed:** Ensure per-frame compute latency is under 15ms to satisfy MathWorks 50Hz closed-loop control requirement.